

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**

Hoàng Danh Minh – 23020292

Chữ Hiệp Nghĩa – 23020298

Nguyễn Văn Phan - 23020300

XÂY DỰNG GAME CỜ CARO AI

BÁO CÁO BÀI TẬP LỚN

Giảng viên: TS. Nguyễn Phương Thái

Ngành: Kỹ thuật điều khiển và Tự động hóa

Mã học phần: 2526II_INT3401_1

HÀ NỘI - 2025

Mục lục

1. Giới thiệu bài toán	3
1.1. Mô tả bài toán cờ Caro	3
1.2. Luật chơi và điều kiện thắng	3
2. Biểu diễn bài toán	4
2.1. Cách biểu diễn trạng thái bàn cờ	4
2.2. Cách sinh nước đi hợp lệ	4
2.3. Cách kiểm tra trạng thái kết thúc	5
3. Thuật toán tìm kiếm	5
3.1. Thuật toán Minimax	5
3.2. Thuật toán Alpha-Beta Pruning	5
3.3. Hàm đánh giá trạng thái	8
3.4. Các cơ chế tối ưu bổ sung	10
4. Thực nghiệm và đánh giá	10
4.1. Thiết kế các trạng thái thử nghiệm	10
4.2. Bảng kết quả thực nghiệm	11
4.3. Nhận xét về số trạng thái đã xét và thời gian chạy	16
4.4. Nhận xét về ảnh hưởng của độ sâu tìm kiếm	18
5. Kết luận	19
5.1. Ưu điểm và hạn chế	19
5.2. Hướng phát triển	20
GITHUB	20
Tài liệu tham khảo	21

1. Giới thiệu bài toán

1.1. Mô tả bài toán cờ Caro

Cờ Caro là trò chơi hai người đấu luân phiên, có tính đối kháng rõ rệt, phù hợp để áp dụng các thuật toán tìm kiếm trong môn Trí tuệ nhân tạo. Mỗi trạng thái bàn cờ tương ứng với một node trong không gian tìm kiếm, mỗi nước đánh là một hành động sinh trạng thái kế tiếp, và AI ra quyết định bằng cách dự đoán phản ứng của đối thủ.

Chương trình được xây dựng với giao diện đồ họa pygame, hỗ trợ hai chế độ AI là Minimax và Alpha-Beta Pruning, kèm hàm đánh giá heuristic để xử lý các trạng thái chưa kết thúc khi tìm kiếm bị giới hạn độ sâu.

Project đáp ứng cả ba mức yêu cầu: Level 1 cài đặt Minimax, Level 2 cài đặt Alpha-Beta Pruning, Level 3 có bộ trạng thái kiểm thử và bảng kết quả thực nghiệm so sánh hai thuật toán.

1.2. Luật chơi và điều kiện thắng

Bàn cờ được biểu diễn dưới dạng ma trận vuông 11×11 , mỗi ô nhận một trong ba giá trị: **rỗng**, quân **X** hoặc quân **O**. Hai bên đánh luân phiên vào các ô trống; một nước đi hợp lệ khi ô được chọn nằm trong phạm vi bàn cờ và chưa có quân.

Điều kiện thắng là có 4 quân liên tiếp theo hàng ngang, hàng dọc, đường chéo chính hoặc đường chéo phụ. Chương trình không áp dụng luật chặn hai đầu — chỉ cần đủ 4 quân liên tiếp là thắng, bất kể hai đầu chuỗi mở hay bị chặn. Nếu bàn cờ lấp đầy mà không bên nào đạt điều kiện trên, kết quả là hòa.

2. Biểu diễn bài toán

2.1. Cách biểu diễn trạng thái bàn cờ

Bàn cờ được biểu diễn bằng danh sách hai chiều trong Python, với ba giá trị: **EMPTY = 0** (ô trống), **PLAYER_X = 1** (quân X), **PLAYER_O = 2** (quân O).

	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	2	0	0	0	0	0	0	0
5	0	0	0	0	2	0	0	0	0	0	0
6	0	0	0	0	0	1	0	0	0	0	0
7	0	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	0
10	0	0	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0	0

Ma trận cờ

	1	2	3	4	5	6	7	8	9	10	11
1											
2											
3											
4				O							
5					O						
6						X					
7				X							
8											
9											
10											
11											

Ảnh bàn cờ hiển thị

2.2. Cách sinh nước đi hợp lệ

Chương trình có hai hàm sinh nước đi:

- ***get_valid_moves()*** — trả về toàn bộ ô trống trên bàn cờ (11x11).
- ***get_candidate_moves()*** — chỉ lấy các ô trống nằm trong bán kính **SEARCH_RADIUS** quanh các quân đã đánh, nhằm giảm hệ số phân nhánh và cho phép AI tìm kiếm sâu hơn.

Nếu bàn chưa có quân, ***get_candidate_moves()*** trả về ô trung tâm. Khi đã có quân, hàm ưu tiên các ô gần cụm quân và gần trung tâm bàn cờ.

2.3. Cách kiểm tra trạng thái kết thúc

Sau mỗi nước đi, hàm ***check_winner()*** duyệt toàn bộ bàn cờ, tại mỗi ô có quân thì đếm số quân liên tiếp theo bốn hướng: ngang, dọc, chéo chính và chéo phụ. Nếu đếm được $\geq \text{WIN_LENGTH}$ quân liên tiếp thì bên đó thắng.

Hàm ***get_game_result()*** tổng hợp kết quả: trả về **X_WIN**, **O_WIN**, **DRAW** hoặc **ONGOING** tương ứng. Việc tách riêng hai hàm này giúp cả giao diện lẫn thuật toán AI dùng chung một bộ luật, tránh sai lệch giữa luật hiển thị và luật tìm kiếm.

3. Thuật toán tìm kiếm

3.1. Thuật toán Minimax

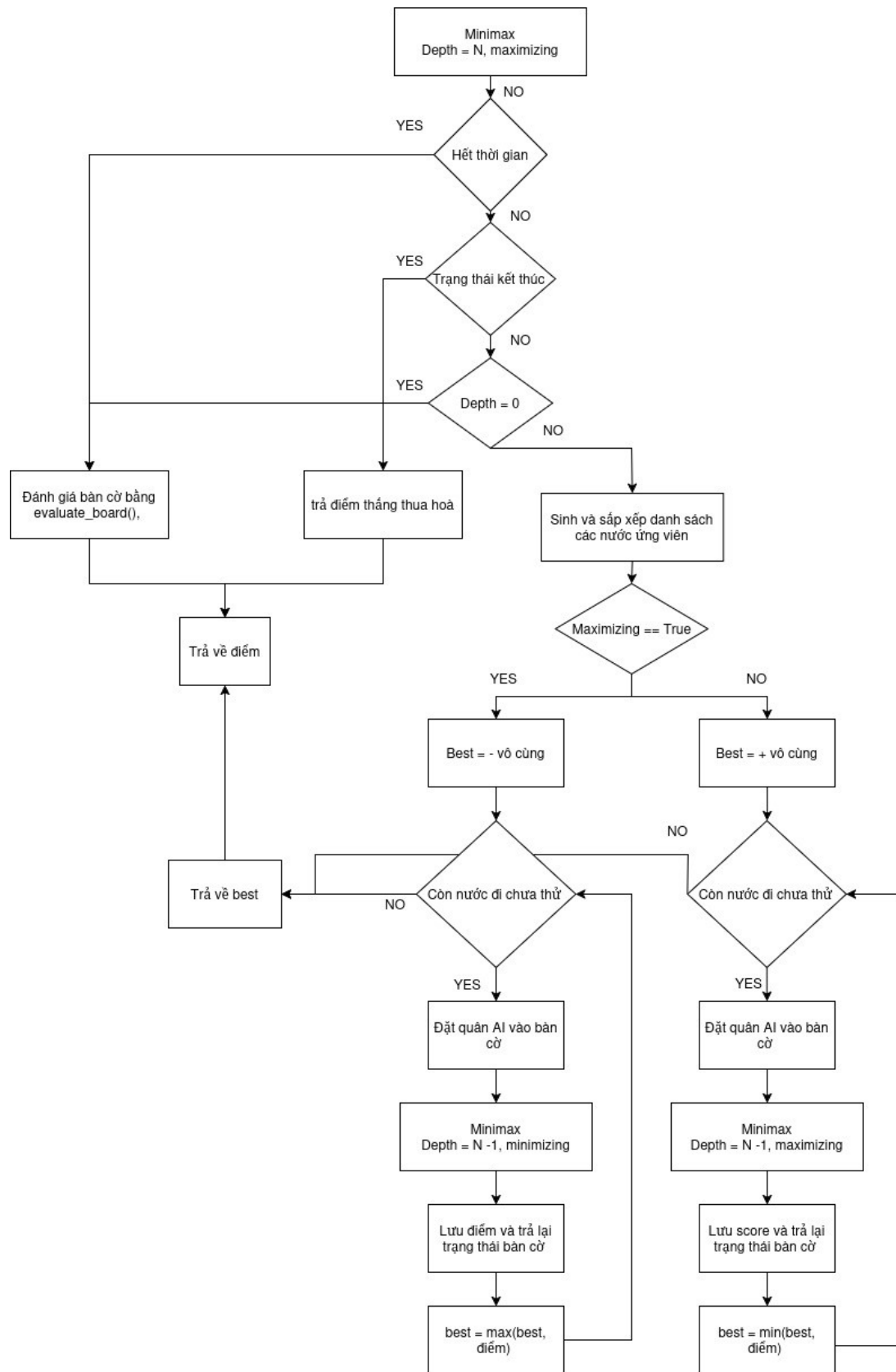
Minimax là thuật toán tìm kiếm đối kháng cho trò chơi hai người tổng bằng không. AI đóng vai MAX — luôn chọn nước có điểm lớn nhất; người chơi đóng vai MIN — được giả định sẽ chọn nước làm điểm của AI nhỏ nhất. Cách mô hình hóa này giúp AI dự đoán phản ứng tốt nhất của đối thủ thay vì chỉ nhìn nước đi hiện tại.

Do không gian trạng thái của Caro rất lớn, Minimax được giới hạn độ sâu. Nếu gặp trạng thái kết thúc thì trả điểm thắng/thua/hòa; nếu đạt độ sâu giới hạn thì gọi ***evaluate_board()*** để ước lượng lợi thế. Mỗi lần xét một trạng thái, bộ đếm nodes tăng lên để phục vụ thống kê. Nếu hết thời gian, thuật toán trả về điểm hiện tại và đánh dấu timeout.

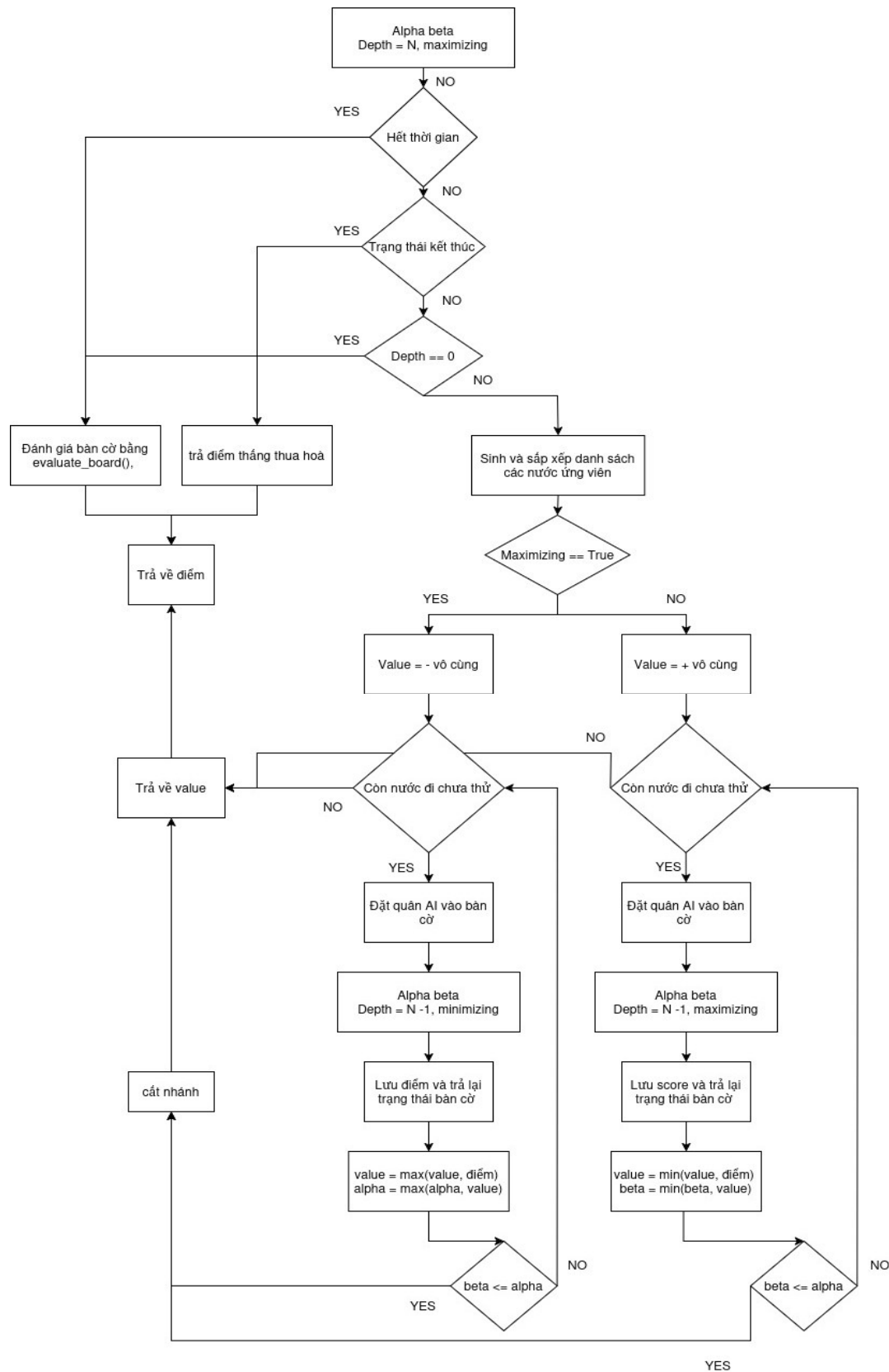
3.2. Thuật toán Alpha-Beta Pruning

Alpha-Beta Pruning là cải tiến của Minimax, duy trì hai biến alpha và beta để loại bỏ các nhánh không ảnh hưởng đến kết quả. **alpha** là giá trị tốt nhất MAX có thể đảm bảo, **beta** là giá trị tốt nhất MIN có thể đảm bảo. Khi **beta** $\leq$ **alpha**, nhánh hiện tại bị cắt bỏ vì không thể thay đổi quyết định ở tầng trên.

Trong project, Alpha-Beta dùng cùng danh sách nước ứng viên, cùng hàm đánh giá và cùng độ sâu với Minimax, đảm bảo kết quả thực nghiệm so sánh công bằng. Hiệu quả cắt nhánh phụ thuộc vào thứ tự xét nước đi — nước tốt được xét sớm giúp **alpha** và **beta** siết chặt nhanh hơn, từ đó cắt được nhiều nhánh hơn.



Lưu đồ thuật toán mini max



Lưu đồ thuật toán alpha beta

3.3. Hàm đánh giá trạng thái

Do Minimax và Alpha-Beta bị giới hạn độ sâu, AI cần hàm ước lượng lợi thế khi chưa đến trạng thái kết thúc. Hàm *evaluate_board()* tính điểm dựa theo góc nhìn của AI và người chơi sau đó cộng tất cả lại: dương là có lợi cho AI, âm là có lợi cho người chơi.

Trước hết, *evaluate_board()* kiểm tra trạng thái thắng/thua trực tiếp. Nếu AI đã thắng, hàm trả về WIN_SCORE = 1_000_000; nếu người chơi đã thắng, hàm trả về -WIN_SCORE. Trường hợp ván cờ chưa kết thúc, chương trình sẽ tính điểm riêng cho AI và người chơi dựa trên các tiêu chí heuristic.

Công thức tổng quát: **Total = ai_score - human_score * DEFENSE_WEIGHT**

Các tiêu chí chấm điểm:

- **Window score:** xét các đoạn độ dài WIN_LENGTH để phát hiện mẫu gần thắng (ví dụ 3 quân + 1 ô trống).
- **Chain score:** đánh giá độ dài chuỗi quân thật sự nối nhau và số đầu mở.
- **Winning move score:** đếm số ô mà một bên có thể đánh để thắng ngay lượt sau; từ hai Winning move trở lên được coi là nước đôi nguy hiểm.
- **Center bonus:** cộng điểm nhẹ cho quân gần trung tâm vì có nhiều hướng phát triển hơn.
- **Trọng số phòng thủ:** điểm nguy hiểm của đối thủ được nhân DEFENSE_WEIGHT để AI ưu tiên chặn trước khi tấn công.

Tiêu chí	Trường hợp	Điểm
Win /Lose		+1_000_000 / -1_000_000
Window score	3 ta, 0 địch	+ 140_000
	2 ta, 0 địch	+ 3_200
	1 ta, 0 địch	+ 500
	Các trường hợp còn lại	+ 0
Chain score	Chuỗi 3 có 2 đầu mở (. X X X .)	+ 230_000
	Chuỗi 3 có 1 đầu mở (O X X X .)	+ 100_000
	Chuỗi 2 có 2 đầu mở (. X X .)	+2_500
	Chuỗi 2 có 1 đầu mở (O X X .)	+1_800
	Chuỗi 1 có 2 đầu mở (. X .)	+ 120
	Chuỗi 1 có 1 đầu mở (O X .)	+ 35
Center bonus	11x11 thì tâm ở (6,6) điểm (5,5) sẽ có distance = 2 (theo khoảng cách)	+ 18 – (distance * 2)
Winning move score (phía AI)	1 chỗ đặt vào là thắng	+ 95_000
	>= 2 chỗ đặt vào là thắng	+ 260_000 * số chỗ
Winning move score (phía Player)	1 chỗ đặt vào là thắng	+ 240_000
	>= 2 chỗ đặt vào là thắng	+ 520_000 * số chỗ

Như vậy, ta có **ai_score = Window score + Chain score + Center bonus + Winning move score (phía AI)**, phía **human_score** tương tự.

Ví dụ:

$$\mathbf{ai_score = 100,000}$$

$$\mathbf{human_score = 80,000}$$

$$\mathbf{Total = 100,000 - 80,000 * 1.4 = 100,000 - 112,000 = -12,000}$$

Mặc dù AI có điểm tấn công cao hơn, nhưng vì có trọng số phòng thủ, trạng thái này vẫn bị đánh giá là tệ cho AI khiến AI sẽ phải chọn phòng thủ thay vì tiếp tục tấn công mù quáng.

3.4. Các cơ chế tối ưu bổ sung

Ngoài Minimax và Alpha-Beta cơ bản, chương trình bổ sung một số cơ chế để AI hoạt động hiệu quả hơn trong thực tế:

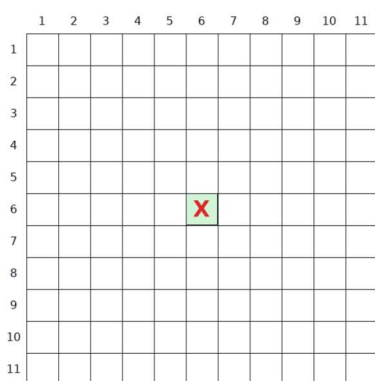
- **Luật thắng/chặn ngay:** trước khi tìm sâu, AI kiểm tra nước thắng ngay; nếu không có thì kiểm tra nước người chơi sắp thắng để chặn. Cả hai trường hợp đều trả kết quả ngay mà không cần duyệt cây.
- **Sắp xếp nước đi:** *tactical_move_score()* chấm điểm nhanh từng ứng viên theo tấn công, phòng thủ, khả năng tạo và chặn fork, giúp Alpha-Beta cắt nhánh hiệu quả hơn.
- **Iterative deepening:** AI tìm từ độ sâu 1 rồi tăng dần, lưu lại nước tốt nhất ở độ sâu hoàn thành sâu nhất. Nếu hết thời gian thì vẫn có nước dự phòng.
- **Giới hạn thời gian và vùng tìm kiếm:** AI_TIME_LIMIT tránh game bị treo; SEARCH_RADIUS và MAX_CANDIDATE_MOVES giảm số nước ứng viên ở mỗi tầng.
- **Xử lý anti-fork:** các hàm *count_win_moves()*, *count_fork_creating_moves()* giúp AI nhận ra và ưu tiên chặn các nước tạo nhiều mối đe dọa cùng lúc.

4. Thực nghiệm và đánh giá

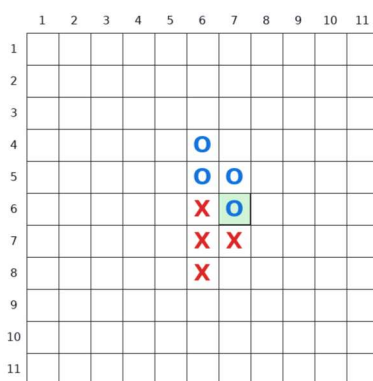
4.1. Thiết kế các trạng thái thử nghiệm

Chuẩn bị sáu trạng thái kiểm thử, bao gồm các tình huống đặc trưng trong ván cờ: đầu ván, giữa ván, thắng ngay, cần phòng thủ, hai bên cùng tấn công và trạng thái có nhiều nhánh. Minimax và Alpha-Beta được chạy trên cùng trạng thái, cùng độ sâu và cùng hàm đánh giá để đảm bảo kết quả so sánh công bằng.

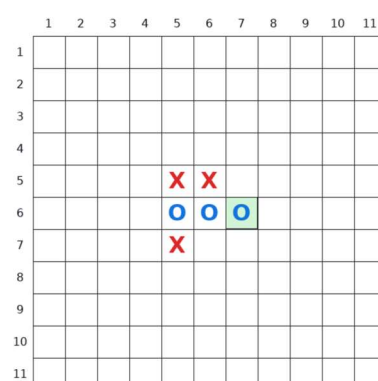
Trạng thái	Mô tả
Đầu ván	Bàn cờ rỗng
Giữa ván	Hai bên đã có nhiều quân ở trung tâm
AI có thể thắng ngay	AI có 3 quân liên tiếp
Người sắp thắng	Người chơi có 3 quân liên tiếp
Hai bên cùng tấn công	Cả hai bên đều có thể phát triển
Nhiều nhánh	Có nhiều vùng quân khác nhau



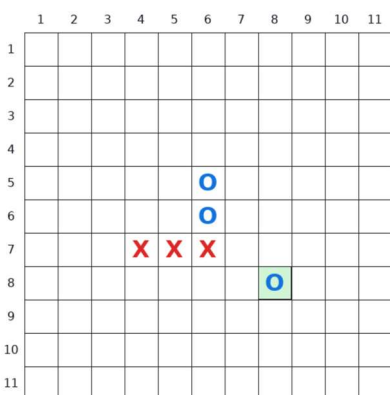
Đầu ván



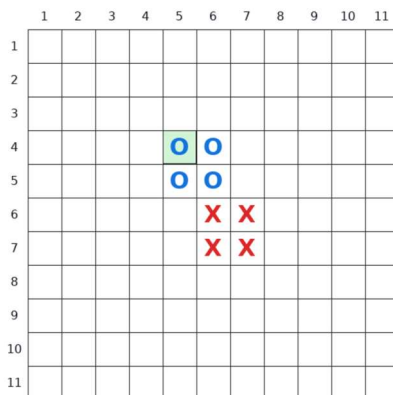
Giữa ván



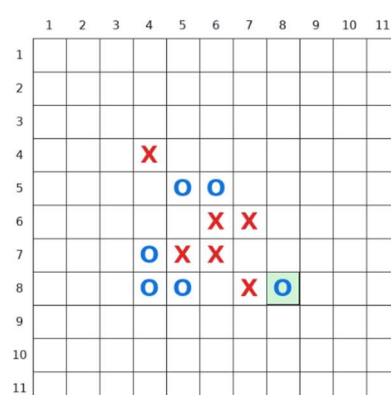
AI sắp thắng (AI = O)



Người sắp thắng (Người = X)



Hai bên cùng tấn công



Nhiều nhánh

4.2. Bảng kết quả thực nghiệm

Bảng dưới đây được lấy từ kết quả file thực nghiệm trên project đã nộp.

Đang chạy trạng thái: Nhiều nhánh - Alpha-Beta...

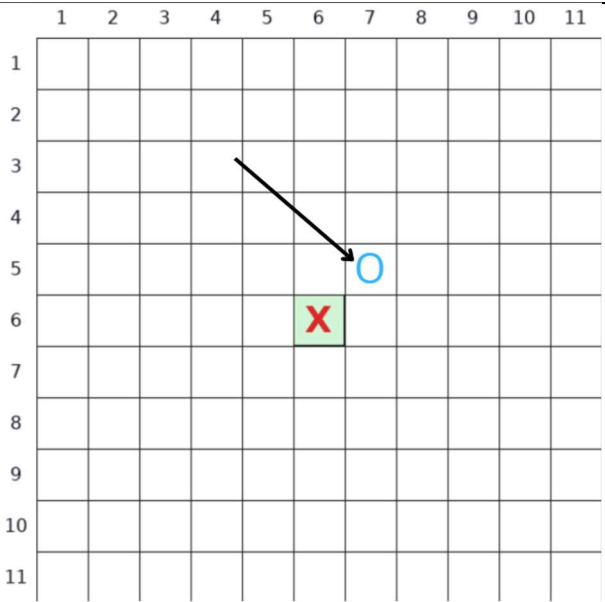
Trạng thái Ghi chú	Thuật toán	Depth	Nước đi	Điểm	Nodes	Cutoffs	Time(s)	Complete
Đầu ván	Minimax	3	(4, 6)	-3932	17136	0	304.500654	True
Tìm kiếm	Alpha-Beta	3	(4, 6)	-3932	1849	86	58.006259	True
Đầu ván	Minimax	3	(8, 5)	-659283	0	0	0.000709	True
Tìm kiếm	Alpha-Beta	3	(8, 5)	-659283	0	0	0.000707	True
Giữa ván	Minimax	3	(5, 3)	1000000	0	0	0.000081	True
AI chặn thắng ngay	Alpha-Beta	3	(5, 3)	1000000	0	0	0.000077	True
Giữa ván	Minimax	3	(6, 6)	-2094116	0	0	0.000523	True
AI chặn thắng ngay	Alpha-Beta	3	(6, 6)	-2094116	0	0	0.000523	True
Người sắp thắng	Minimax	3	(4, 6)	1000000	39102	0	266.973455	True
AI chặn thắng ngay	Alpha-Beta	3	(4, 6)	1000000	1942	74	21.545143	True
Người sắp thắng	Minimax	3	(5, 4)	1000000	39102	0	301.995116	True
AI chặn thắng ngay	Alpha-Beta	3	(5, 4)	1000000	1696	73	22.928943	True

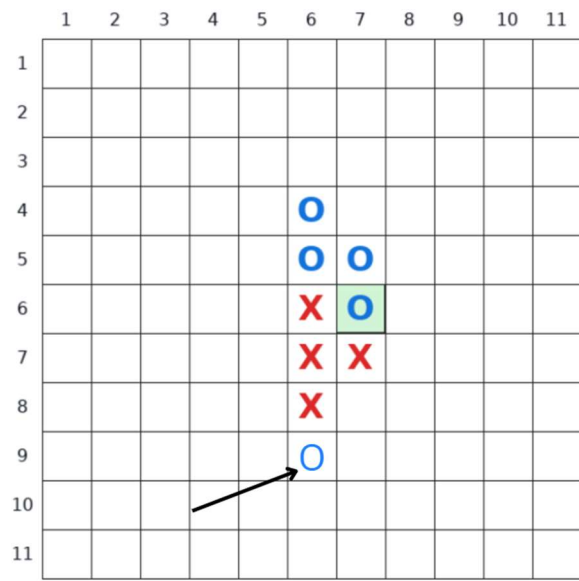
Ảnh kết quả chạy thực nghiệm

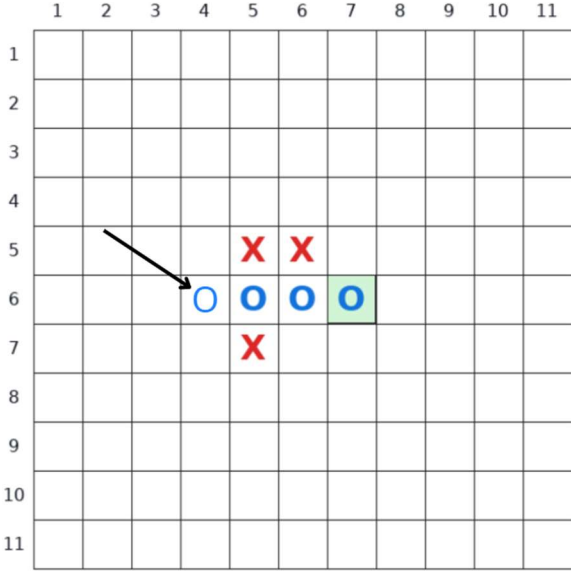
Lưu ý: do mảng python bắt đầu từ 0 nên ta phải cộng 1 ở tất cả ô vị trí. Bảng kết quả dưới đã được cộng đầy đủ

Các giá trị thời gian có thể thay đổi tùy cấu hình máy, nhưng vẫn phản ánh được cách chương trình ghi nhận số node, số nhánh cắt và nước đi được chọn.

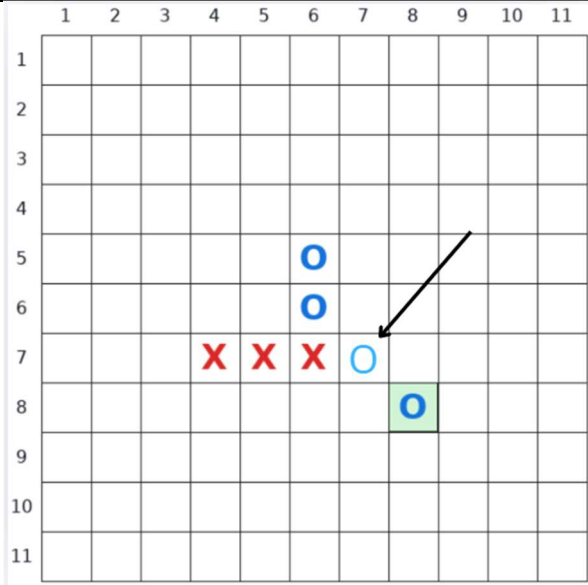
Đầu ván			
Depth	Minimax	Alpha-Beta	Vị trí đặt
2	Nước: (5, 7) Điểm: -25849 Nodes: 576 Time: 5.9608s	Nước: (5, 7) Điểm: -25849 Nodes: 107 Cuts: 23 Time: 5.0651s	
3	Nước: (5, 7) Điểm: -3932 Nodes: 17136	Nước: (5, 7) Điểm: -3932 Nodes: 1849	

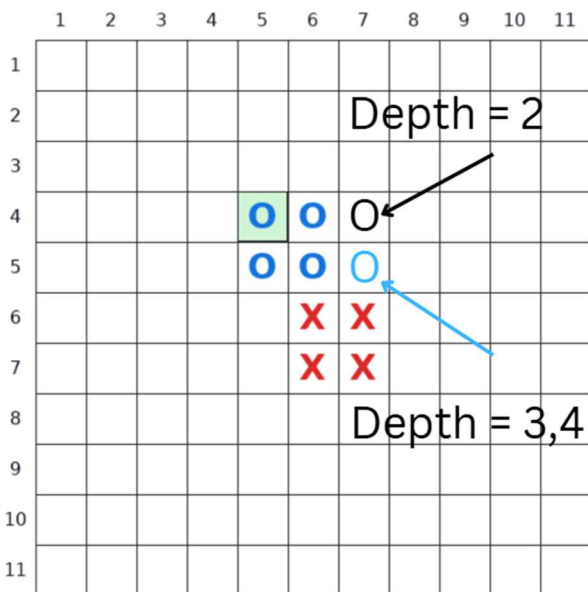
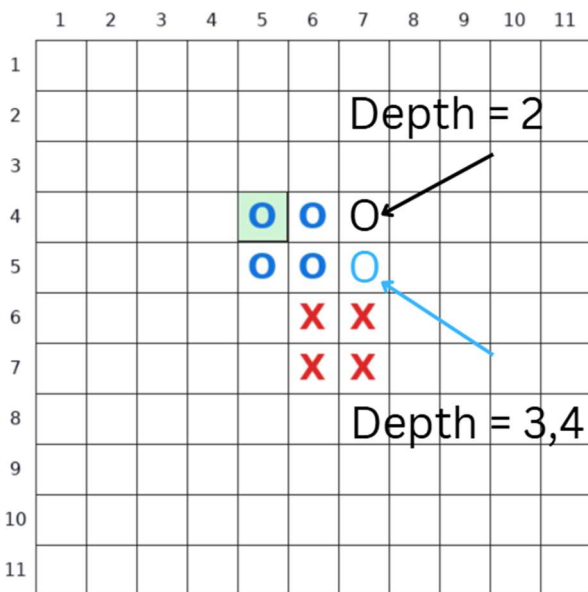
	Time: 304.5006s	Cuts: 86 Time: 58.0063s	 (5,7)
4	Nước: (5, 7) Điểm: -658101 Nodes: 60894 Time: 999.076s (có thể lớn hơn do bị giới hạn time)	Nước: (5, 7) Điểm: -658101 Nodes: 6041 Cuts: 1440 Time: 532.6557s	

Giữa ván			
Depth	Minimax	Alpha-Beta	Vị trí đặt
2	Nước: (9, 6) Điểm: -659283 Nodes: 0 Time: 0.00064s	Nước: (9, 6) Điểm: -659283 Nodes: 0 Cuts: 0 Time: 0.00064s	 (9,6)
3	Nước: (9, 6) Điểm: -659283 Nodes: 0 Time: 0.00077s	Nước: (9, 6) Điểm: -659283 Nodes: 0 Cuts: 0 Time: 0.00068s	
4	Nước: (9, 6) Điểm: -659283 Nodes: 0 Time: 0.00078s	Nước: (9, 6) Điểm: -659283 Nodes: 0 Cuts: 0 Time: 0.00083s	

AI sắp thắng			
Depth	Minimax	Alpha-Beta	Vị trí đặt
2	Nước: (6, 4) Điểm: 1000000 Nodes: 0 Time: 0.0001s	Nước: (6, 4) Điểm: 1000000 Nodes: 0 Cuts: 0 Time: 0.0001s	(6,4) 
3	Nước: (6, 4) Điểm: 1000000 Nodes: 0 Time: 0.0001s	Nước: (6, 4) Điểm: 1000000 Nodes: 0 Cuts: 0 Time: 0.0001s	
4	Nước: (6, 4) Điểm: 1000000 Nodes: 0 Time: 0.0001s	Nước: (6, 4) Điểm: 1000000 Nodes: 0 Cuts: 0 Time: 0.0001s	

Người sắp thắng			
Depth	Minimax	Alpha-Beta	Vị trí đặt
2	Nước: (7, 7) Điểm: -2094116 Nodes: 0 Time: 0.0001s	Nước: (7, 7) Điểm: -2094116 Nodes: 0 Cuts: 0 Time: 0.0001s	
3	Nước: (7, 7) Điểm: -2094116 Nodes: 0 Time: 0.0007s	Nước: (7, 7) Điểm: -2094116 Nodes: 0 Cuts: 0 Time: 0.0005s	

4	Nước: (7, 7) Điểm: -2094116 Nodes: 0 Time: 0.0006s	Nước: (7, 7) Điểm: -2094116 Nodes: 0 Cuts: 0 Time: 0.0008s	 (7,7)

2 bên cùng tấn công			
Depth	Minimax	Alpha-Beta	Vị trí đặt
2	Nước: (4, 7) Điểm: -1097400 Nodes: 1302 Time: 7.3331s	Nước: (4, 7) Điểm: -1097400 Nodes: 184 Cuts: 39 Time: 5.3288s	 Depth = 2
	Nước: (5, 7) Điểm: 1000000 Nodes: 39102 Time: 266.9734s	Nước: (5, 7) Điểm: 1000000 Nodes: 1942 Cuts: 74 Time: 21.5451s	
4	Nước: (5, 7) Điểm: 1000000 Nodes: 162040 Time: 999s (có thể lớn hơn do	Nước: (5, 7) Điểm: 1000000 Nodes: 2732 Cuts: 1311 Time: 146.5998s	 (4,7) và (5,7)

	bị giới hạn time)		
--	-------------------	--	--

Nhiều nhánh			
Depth	Minimax	Alpha-Beta	Vị trí đặt
2	Nước: (5, 4) Điểm: -1707660 Nodes: 1302 Time: 10.3566s	Nước: (5, 4) Điểm: -1097400 Nodes: 154 Cuts: 40 Time: 7.5577s	(5,4) và (6,5)
3	Nước: (6, 5) Điểm: 1000000 Nodes: 39102 Time: 301.9951s	Nước: (6, 5) Điểm: 1000000 Nodes: 1942 Cuts: 74 Time: 22.9289s	
4	Nước: (6, 5) Điểm: 1000000 Nodes: 131921 Time: 999s (có thể lớn hơn do bị giới hạn time)	Nước: (6, 5) Điểm: 1000000 Nodes: 2906 Cuts: 1307 Time: 150.6267s	

4.3. Nhận xét về số trạng thái đã xét và thời gian chạy

Kết quả thực nghiệm cho thấy Alpha-Beta luôn chọn cùng nước đi với Minimax trong tất cả các trạng thái kiểm thử, nhưng số trạng thái cần xét ít hơn đáng kể. Điều này phù hợp với bản chất của Alpha-Beta Pruning: thuật toán vẫn giữ nguyên nguyên lý chọn nước của Minimax, nhưng sử dụng điều kiện $\beta \leq \alpha$ để cắt bỏ các nhánh không còn ảnh hưởng đến quyết định cuối cùng.

Bảng phân tích so sánh Minimax và alphabeta của các trạng thái depth khác nhau:

	Depth = 2	Depth = 3	Depth = 4
Đầu ván	Giảm node: 469 (81.42%) Cutoff: 23 Giảm time: 15.03%	Giảm node: 15287 (89.42%) Cutoff: 86 Giảm time: 80.95%	Giảm node: 60997 (90.99%) Cutoff: 1440 Giảm time: 46.69%
2 bên cùng tấn công	Giảm node: 1118 (85.87%) Cutoff: 39 Giảm time: 27.33%	Giảm node: 37160 (95.03%) Cutoff: 74 Giảm time: 91.93%	Giảm node: 159308 (98.31%) Cutoff: 1311 Giảm time: 85.3%
Nhiều nhánh	Giảm node: 1148 (88.17%) Cutoff: 40 Giảm time: 24.07%	Giảm node: 37406 (95.66%) Cutoff: 73 Giảm time: 92.41%	Giảm node: 129015 (97.8%) Cutoff: 1307 Giảm time: 84.92%

Lưu ý: các trạng thái như “AI sắp thắng”, “Giữa ván” và “người sắp thắng” đều có nodes = 0. Nguyên nhân là chương trình xử lý các tình huống bằng luật tối ưu bổ sung trước khi gọi Minimax hoặc Alpha-Beta. Vì vậy, thời gian ở các dòng này rất nhỏ và không nên dùng để đánh giá hiệu năng thật sự của hai thuật toán. Chúng chỉ cho thấy AI có bước kiểm tra nhanh để tránh tìm kiếm không cần thiết.

Ở depth = 4, do giới hạn thời gian chạy là 999s nên minimax không thể đạt đến độ sâu tận cùng, dữ liệu trên là alpha beta so sánh với minimax ở 999s cho nên dữ liệu thực tế có thể khác rất nhiều.

Ở depth = 2, với các trạng thái thật sự phải tìm kiếm, Alpha-Beta giảm trung bình 85.42% số node và giảm trung bình 15.03% thời gian chạy so với Minimax. Ví dụ ở trạng thái “Đầu ván”, Minimax xét 576 node, trong khi Alpha-Beta chỉ xét 107 node. Ở trạng thái “Nhiều nhánh”, Minimax xét 1302 node, còn Alpha-Beta chỉ xét 154 node. Điều này cho thấy Alpha-Beta hiệu quả hơn rõ rệt ngay cả ở độ sâu nhỏ.

Ở $\text{depth} = 3$, sự khác biệt còn rõ hơn. Alpha-Beta giảm trung bình 93.37% số node và 88.43% thời gian chạy ở các trạng thái cần tìm kiếm. Ví dụ, ở trạng thái “Hai bên cùng tấn công”, Minimax xét 39102 node trong khoảng 266.97 giây, còn Alpha-Beta chỉ xét 1942 node trong khoảng 21.55 giây. Ở trạng thái “Nhiều nhánh”, Minimax cũng xét 39102 node, trong khi Alpha-Beta chỉ xét 1696 node. Đây là minh chứng rõ ràng rằng Alpha-Beta giúp giảm mạnh chi phí duyệt cây khi số nhánh tăng lên.

Ở $\text{depth} = 4$, Minimax bắt đầu gặp giới hạn rất rõ về thời gian. Một số trạng thái như “Đầu ván”, “Hai bên cùng tấn công” và “Nhiều nhánh” không hoàn thành trong giới hạn thời gian 999 giây. Trong khi đó, Alpha-Beta vẫn hoàn thành các trạng thái này nhờ cắt nhánh, dù thời gian chạy vẫn tăng khá nhiều. Ở $\text{depth} 4$, Alpha-Beta giảm trung bình 95.70% số node và 72.31% thời gian ở các trạng thái cần tìm kiếm. Điều này cho thấy khi độ sâu tăng, Minimax thuần trở nên rất tốn kém, còn Alpha-Beta vẫn có khả năng kiểm soát số trạng thái cần xét tốt hơn.

4.4. Nhận xét về ảnh hưởng của độ sâu tìm kiếm

Độ sâu tìm kiếm ảnh hưởng rất lớn đến số trạng thái cần xét và thời gian chạy. Khi tăng depth , thuật toán phải mô phỏng thêm nhiều lượt đi tiếp theo. Do mỗi trạng thái có nhiều nước đi ứng viên, số node trong cây tìm kiếm tăng rất nhanh theo độ sâu. Đây là lý do Minimax ở $\text{depth} 2$ còn chạy được khá nhanh, nhưng đến $\text{depth} 3$ thời gian đã tăng mạnh, và đến $\text{depth} 4$ thì một số trạng thái không hoàn thành trong giới hạn thời gian.

Cụ thể, ở trạng thái “Đầu ván”, Minimax tăng từ 576 node ở $\text{depth} 2$ lên 17136 node ở $\text{depth} 3$, rồi lên 67038 node ở $\text{depth} 4$ nhưng chưa hoàn thành tìm kiếm. Trong khi đó, Alpha-Beta tăng từ 107 node ở $\text{depth} 2$ lên 1849 node ở $\text{depth} 3$ và 6041 node ở $\text{depth} 4$. Như vậy, cả hai thuật toán đều bị ảnh hưởng khi tăng depth , nhưng Alpha-Beta tăng chậm hơn nhiều nhờ khả năng cắt nhánh.

Với các trạng thái phức tạp hơn như “Hai bên cùng tấn công” và “Nhiều nhánh”, tác động của depth càng rõ. Ta thấy $\text{depth} 2$ có sự khác biệt ở vị trí điểm so với $\text{depth} 3$ và 4, khi ta nhìn thế cờ thì đây không phải là nước đi tốt nhất nhưng dù sao thì O cũng sẽ thắng. Ở $\text{depth} 2$, Minimax xét 1302 node ở mỗi trạng thái này. Sang $\text{depth} 3$, số node

tăng lên 39102. Đến depth 4, Minimax xét hơn 130000 đến 160000 node nhưng vẫn không hoàn thành trong giới hạn thời gian. Đây là hiện tượng bùng nổ tổ hợp trong cây tìm kiếm.

Tăng độ sâu giúp AI nhìn trước nhiều lượt hơn, từ đó có khả năng phát hiện các tình huống chiến thuật sâu hơn như tạo thế thắng, chặn nước nguy hiểm hoặc phản công sau khi đối thủ đáp trả. Tuy nhiên, độ sâu càng lớn thì chi phí tính toán càng tăng nhanh. Vì vậy, trong thực tế, cần cân bằng giữa chất lượng nước đi và thời gian xử lý. Kết quả cho thấy depth 3 là mức tương đối phù hợp để so sánh vì cả Minimax và Alpha-Beta đều hoàn thành, trong khi depth 4 đã quá nặng đối với Minimax ở một số trạng thái.

Từ kết quả thực nghiệm có thể kết luận rằng Alpha-Beta đặc biệt hiệu quả khi độ sâu tăng. Ở depth 2, Alpha-Beta đã giảm trung bình 85.42% số node; ở depth 3 tăng lên 93.37%; và ở depth 4 đạt 95.70% ở các trạng thái cần tìm kiếm. Điều này cho thấy Alpha-Beta càng phát huy tác dụng khi cây tìm kiếm lớn hơn.

5. Kết luận

5.1. Ưu điểm và hạn chế

Ưu điểm

- Project có cấu trúc rõ ràng, tách biệt giao diện, luật chơi, AI, đánh giá và file thực nghiệm.
- Đáp ứng đúng luật Caro 4 quân liên tiếp, không xét chặn hai đầu theo yêu cầu đề bài.
- Có cả Minimax và Alpha-Beta pruning, dùng chung hàm đánh giá để so sánh công bằng.
- Có thống kê nodes, cutoffs, thời gian, nước tốt nhất và độ sâu hoàn thành.
- Hàm đánh giá tương đối giàu tiêu chí, có xét tấn công, phòng thủ, threat, nước đôi và trung tâm.
- Có iterative deepening và giới hạn thời gian để AI không bị treo khi cây tìm kiếm lớn.
- Kết hợp giữa tìm kiếm cây trò chơi và luật chiến thuật nhanh.

Hạn chế

- Khi bàn cờ lớn hoặc tăng độ sâu, số trạng thái vẫn tăng nhanh, khiến AI có thể chạy lâu.
- Hàm đánh giá là heuristic nên chưa thể đảm bảo tối ưu tuyệt đối trong mọi thế cờ.
- Project chưa dùng transposition table hoặc Zobrist hashing nên các trạng thái trùng lặp chưa được tái sử dụng.
- File main.py còn khá dài vì gán nhiều phần giao diện; nếu phát triển tiếp có thể tách UI thành nhiều file nhỏ hơn.
- Chưa áp dụng Machine Learning hoặc Reinforcement Learning.
- Giao diện và chức năng phụ còn đơn giản.

5.2 Hướng phát triển

Nếu tiếp tục phát triển, chương trình có thể cải thiện theo bốn hướng chính:

- **Cải tiến hàm đánh giá:** giúp AI nhận diện tốt hơn các thế cờ nguy hiểm như nước đôi, nước mồi, chuỗi mở hai đầu và các tình huống vừa chặn vừa phản công.
- **Tối ưu hiệu năng tìm kiếm:** cải thiện thứ tự duyệt nước đi, giới hạn nước ứng viên thông minh hơn và bổ sung bảng ghi nhớ trạng thái (transposition table) để tránh tính toán lặp.
- **Học từ ván đã chơi:** AI lưu lại các chuỗi nước đi dẫn đến thất bại để giảm điểm hoặc né tránh khi gặp lại trạng thái tương tự.
- **Áp dụng học máy:** tự điều chỉnh trọng số hàm đánh giá thay vì thiết lập thủ công, giúp AI học được các thế trận thường dẫn đến thắng hoặc thua qua nhiều ván chơi.

Ngoài ra, phần thực nghiệm có thể mở rộng với nhiều trạng thái kiểm thử và nhiều mức độ sâu hơn để đánh giá chính xác hiệu quả của từng thuật toán.

GITHUB

link github: https://github.com/t-o-n-e-r-y/23020292_23020298_23020300_CaroAI

Tài liệu tham khảo

- <https://github.com/husus/gomokuAI-py>
- https://github.com/MonHauVD/Caro_AI
- Kiến thức bài giảng môn Trí Tuệ Nhân Tạo trên lớp
- Tài liệu chính thức của thư viện pygame.